

Assignment 2 Advance

```
1. class EventProcessor:
public class EventProcessor {
    private Database database;
    private Dashboard dashboard;
    private Logger logger;
    public EventProcessor(Database database, Dashboard dashboard, Logger logger) {
        this.database = database;
        this.dashboard = dashboard;
        this.logger = logger;}

    // One giant method that processes all types based on event.type
    public void process(Event e) {
        // validate the new event!
        if (e.getPayload() == null || e.getPayload().isEmpty()) {
            System.out.println("Invalid event");
            return;}
        // notify various subsystems with new event.
        dashboard.updateMetrics(e);
        logger.log(e);
        // transform (lots of if/else)
        String data = e.getPayload();
        if (e.isEncrypt()) {
            // naive encryption
            data = "ENC(" + data + ")"; }
        if (e.isCompress()) {
            data = "CMP(" + data + ")";}
        if (e.isAddMetadata()) {
            data = "META(" + e.getMetadata() + ")::" + data;
        }
        // persist
        e.setId(System.currentTimeMillis() + "-" + Math.abs(data.hashCode()));
        database.save(e.getId(), data);
        // extra behavior depending on type
        if ("SECURITY".equals(e.getType())) {
            // duplicate security handling across code
            System.out.println("[SECURITY] extra analysis for " + e.getId());
            // directly send to security monitor (hard-coded)
            System.out.println("[SecurityMonitor] alert for " + e.getId());
        } else if ("USER".equals(e.getType())) {
            System.out.println("[USER] user-specific step for " + e.getId());
        } else if ("SYSTEM".equals(e.getType())) {
            System.out.println("[SYSTEM] system audit log " + e.getId());}}}
}
```

The Problems:

Violate SRP: This class doing many operations (validation, type-specific behavior, notification, transformation, and persistence).

Violate OCP: when add any event (do if statement)

Violate DIP: depends on class Dashboard, class Database, and class Logger.

Large code is difficult to maintain or to read.

code duplication

The solution:

Make an interface and implement it

2. Class event:

Supports SRP one goal but can do instead of String enum to violate error mistake

3. Class Logger, Class Database, Class Dashboard:

Supports SRP one goal , small code and print-only at console(system.out.print)

But improvements can be do interface for each when chang it be easy

UML:

